

Yüksek Performanslı ve Eşzamanlı C++

Genel C++ sözdizimini tekrar etmek yerine, modern C++ ile performans mühendisliği ve concurrency alanına odaklanan özel ders: cache locality, allocation, move, benchmarking, threads, atomics, lock-free düşünce, SIMD ve profiling.

Seviye

Orta -> İleri

Önerilen süre

20-28 saat

Belge yapısı

15 sayfa / ders modülleri + uygulama

Kapsam: Cache | Concurrency | Atomics | Profiling

Amaç: Tek başına çalışılabilir, örnek ve mini uygulamalar içeren ders dokümanı.

MODÜL 0

Özel Konu Haritası: Performans Mühendisliği

Bu dersin ana fikri: "hızlı kod" tahminle değil, ölçüm ve donanım farkındalığıyla üretilir. Önce darboğaz ölçülür, ardından algoritma, bellek erişimi, allocation ve concurrency seçenekleri değerlendirilir.

Bölüm	Odak	Ana soru
3-6	Bellek ve nesne maliyeti	CPU veriye ne kadar hızlı ulaşıyor?
2	Benchmark yöntemi	Ölçüm doğru mu?
7-11	Concurrency	İş nasıl güvenli ve ölçeklenebilir bölünür?
12	SIMD + DOD	Veri düzeni CPU dostu mu?
13-15	Profiling + proje	Darboğaz kanıtlandı mı?

Ön koşul

Temel C++: class, STL, smart pointer, template, RAII ve temel thread kavramlarına aşinalık önerilir.

- Latency ve throughput farkını ölçer.
- Cache locality ve false sharing sorunlarını tanımlar.
- Move semantics ve allocation stratejilerini performans açısından değerlendirir.
- std::jthread, mutex, atomics ve memory order araçlarını doğru yerde kullanır.
- Profilер ve sanitizer çıktılarından optimizasyon hipotezi çıkarır.

MODÜL 1

Ölçmeden Optimize Etme: Latency, Throughput, Benchmark

Performans hedefi önce sayısallaştırılmalıdır. Latency tek bir işin tamamlanma süresidir; throughput birim zamanda tamamlanan iş miktarıdır. Ortalama tek başına yeterli değildir; p95/p99 gecikme özellikle servis ve gerçek zamanlı sistemlerde önemlidir.

Metrik	Örnek
Latency	Bir mesajı işleme: 280 us
Throughput	Saniyede 120k mesaj
p99	İşlerin %99'u 1.8 ms altında
CPU utilization	8 çekirdeğin %72 ortalama kullanımı

MIKRO ÖLÇÜM ISKELETİ

```
auto start = std::chrono::steady_clock::now();
for (int i = 0; i < iterations; ++i) {
    benchmark_target(data);
}
auto end = std::chrono::steady_clock::now();
std::cout << std::chrono::duration<double, std::micro>(end-start).count();
```

Benchmark tuzağı

Warm-up, CPU frekans değişimi, dead-code elimination, I/O ve debug derleme sonuçları ciddi biçimde bozabilir. Kritik ölçümlerde Google Benchmark benzeri altyapı kullanın.

MODÜL 2

CPU Cache ve Locality

Modern CPU, ana bellekten çok daha hızlıdır; bu nedenle performans çoğu zaman aritmetikten çok veri erişimi tarafından belirlenir. Cache line tipik olarak sabit boyutlu bloklarla veri taşır. Ardışık erişim donanım prefetcher için avantajlıdır.

LOCALITY KARŞILAŞTIRMASI

```
// Cache-friendly: contiguous traversal
std::vector<int> values(1'000'000);
long long sum = 0;
for (int v : values) sum += v;

// Pointer chasing can be much slower:
for (Node* n = head; n != nullptr; n = n->next)
    sum += n->value;
```

Kavram	Etkisi
Temporal locality	Yakın zamanda kullanılan verinin tekrar kullanılması
Spatial locality	Komşu veriye ardışık erişim
Cache miss	CPU daha yavaş bellek katmanını bekler
Prefetch	Yaklaşan veriyi önceden getirme

Mini deney

Aynı sayıda integer içeren vector ve linked-list üzerinde toplama benchmarkı yapın. Sonuçları Release derlemede karşılaştırın.

MODÜL 3

Object Layout, Alignment ve False Sharing

Bir struct alanlarının sırası padding miktarını değiştirebilir. Çok threadli kodda ise birbirinden bağımsız sayaçların aynı cache line üzerinde bulunması false sharing yaratabilir: çekirdekler aynı line için sürekli cache coherence trafiği üretir.

ALIGNMENT ÖRNEĞİ

```
struct BadLayout {
    char flag;    // 1 byte
    double value; // alignment may add padding
    int id;
};

struct alignas(64) Counter {
    std::atomic<std::uint64_t> value{0};
};
```

- sizeof ve alignof ile gerçek yerleşimi ölçün.
- Alanları yalnızca boyuta göre yeniden sıralamak ABI/semantik etkiler doğurabilir; ölçmeden değiştirmeyin.
- hardware_destructive_interference_size desteklenen derleyicilerde cache line ayrıştırma için ipucu verebilir.
- False sharing veri yarışı değildir; program doğru çalışırken performans düşebilir.

Teşhis

Thread sayısı arttıkça hızlanmak yerine yavaşlayan basit sayaç kodunda false sharing olasılığını inceleyin.

MODÜL 4

RAII, Move Semantics ve Zero-Copy Düşüncesi

Modern C++ performansında sahiplik ve veri kopyalama maliyeti önemlidir. Move semantics pahalı kaynakların sahipliğini kopyalamadan transfer etmeyi sağlar. Ancak std::move kendisi taşıma yapmaz; nesneyi rvalue olarak işaretleyip uygun overload seçimine izin verir.

MOVE + SPAN

```
std::vector<std::byte> make_packet() {  
    std::vector<std::byte> data(4096);  
    // fill data  
    return data; // NRVO / move  
}  
  
void consume(std::span<const std::byte> bytes);  
auto packet = make_packet();  
consume(packet); // non-owning view, no copy
```

Araç	Performans rolü
std::move	Sahipliği taşıyan overload seçimi
std::span	Kopyasız, sahip olmayan contiguous view
string_view	Kopyasız string görünümü
unique_ptr	Tek sahiplik + deterministik yaşam döngüsü

Yaşam süresi uyarısı

span ve string_view sahip değildir; referans verdikleri veri yok olursa dangling view oluşur.

MODÜL 5

Allocation Maliyeti ve Memory Resource

Sık heap allocation; allocator kilitleri, metadata, cache miss ve parçalanma maliyetleri oluşturabilir. Hot path üzerinde nesne üretimini azaltmak, reserve kullanmak veya arena/pool yaklaşımı değerlendirmek performansı artırabilir.

RESERVE + PMR

```
std::vector<Event> events;  
events.reserve(100000);  
  
std::array<std::byte, 1024 * 1024> buffer;  
std::pmr::monotonic_buffer_resource arena(buffer.data(), buffer.size());  
std::pmr::vector<Event> temp{&arena};  
temp.reserve(10000);
```

- reserve kapasiteyi önceden ayırarak tekrar allocation sayısını azaltabilir.
- pmr polymorphic allocator stratejisini container tipinden ayırır.
- monotonic_buffer_resource toplu ve kısa ömürlü nesnelerde çok hızlı olabilir; bireysel free yapmaz.
- Small-object optimization bazı kütüphane tiplerinde heap kullanımını azaltabilir; implementation detail olabilir.

Mini deney

100k küçük nesneyi default allocator ve monotonic_buffer_resource ile üretip süre + allocation sayısını karşılaştırın.

MODÜL 6

std::thread, std::jthread ve İş Bölme

Thread eklemek otomatik hızlanma sağlamaz. İş yeterince büyük değilse thread oluşturma, scheduling ve synchronization maliyeti kazancı aşar. std::jthread scope sonunda otomatik join yapar ve stop_token ile cooperative cancellation destekler.

JTHREAD ÖRNEĞİ

```
void worker(std::stop_token st, WorkQueue& q) {
    while (!st.stop_requested()) {
        if (auto item = q.try_pop()) process(*item);
        else std::this_thread::yield();
    }
}

std::jthread t(worker, std::ref(queue));
// t destructor requests clean join semantics
```

- CPU-bound işte ideal thread sayısı işin yapısına ve hardware concurrency değerine bağlıdır.
- I/O-bound işlerde farklı strateji gerekebilir.
- Thread affinity bazen cache locality için yararlı olabilir, fakat portability maliyeti vardır.
- Granularity çok küçükse task scheduling overhead baskın hale gelir.

Mini görev

1 milyon öğeyi 1,2,4,8 thread ile işleyip speedup eğrisi çıkarın; seri süreyi baz alın.

MODÜL 7

Mutex, Condition Variable, Semaphore ve Barrier

Synchronization primitive seçimi problemi doğru ifade etmelidir. Mutex paylaşılan invariantı korur. condition_variable bir koşul değişene kadar bekletir. counting_semaphore eşzamanlı kaynak kullanımını sınırlar. barrier belirli sayıda threadin faz sonunda buluşmasını sağlar.

CONDITION VARIABLE KALIBI

```
std::mutex m;  
std::condition_variable cv;  
bool ready = false;  
  
std::unique_lock lock(m);  
cv.wait(lock, [&] { return ready; });  
// invariant protected while lock is held
```

Primitive	İdeal kullanım
mutex	Kısa kritik bölge / paylaşılan invariant
condition_variable	Durum değişimi bekleme
semaphore	Kaynak veya concurrency limiti
barrier/latch	Faz senkronizasyonu

Deadlock önleme

Birden fazla mutex gerekiyorsa tutarlı kilit sırası veya std::scoped_lock kullanın. Kritik bölgeyi kısa tutun ve kilit altında I/O yapmaktan kaçının.

MODÜL 8

Atomics ve C++ Memory Model

std::atomic veri yarışını önleyen atomik operasyonlar sağlar; fakat tüm algoritmayı otomatik olarak doğru yapmaz. Memory order, farklı threadlerin memory operationlarını hangi sıra ve görünürlük garantileriyle gözlemlediğini tanımlar.

ACQUIRE / RELEASE

```
std::atomic<bool> ready{false};
Data data;

// Producer
data = build();
ready.store(true, std::memory_order_release);

// Consumer
if (ready.load(std::memory_order_acquire)) {
    consume(data);
}
```

Order	Genel anlam
relaxed	Sadece atomicity; ek ordering yok
acquire	Sonraki okumalar release öncesini görebilir
release	Önceki yazıları yayınlar
seq_cst	En güçlü, global sıralama modeli

Güvenli varsayılan

Memory order konusunda net kanıtınız yoksa seq_cst veya daha basit synchronization primitive kullanmak genellikle daha güvenlidir.

MODÜL 9

Lock-Free Düşünce, CAS ve ABA Problemi

Lock-free, hiçbir threadin ilerlemesinin başka bir threadin askıda kalmasına tamamen bağlı olmadığı ilerleme garantisidir; wait-free daha güçlüdür. Lock-free kod her zaman mutex kodundan hızlı değildir ve doğruluğu çok daha zordur.

COMPARE-AND-SWAP DÖNGÜSÜ

```
std::atomic<int> value{0};
int expected = value.load();
do {
    // expected is updated on failure
} while (!value.compare_exchange_weak(
    expected, expected + 1,
    std::memory_order_relaxed));
```

- CAS yarış durumunda başarısız olabilir ve tekrar denenir.
- ABA: değer A -> B -> A değiştiğinde yalnız değere bakan CAS aradaki değişimi kaçırabilir.
- Hazard pointers, epoch reclamation veya tagged pointers güvenli memory reclamation için kullanılan ileri tekniklerdir.
- Standart mutex çoğu iş kodunda daha okunabilir, bakım kolay ve yeterince hızlıdır.

Karar ilkesi

Lock-free yapıya ancak profiler synchronization contention gösterdiğinde ve doğruluk/test kapasitesi yeterliyse geçin.

MODÜL 10

Task Parallelism, Futures ve Thread Pool Tasarımı

Uygulamalar genellikle uzun ömürlü threadlerden çok kısa görevler üretir. Thread pool, thread oluşturma maliyetini amorti eder. İş kuyruğu, backpressure, exception aktarımı ve shutdown semantiği tasarımın kritik parçalarıdır.

FUTURE İLE SONUÇ TAŞIMA

```
auto future = std::async(std::launch::async, [] {
    return expensive_compute();
});

// do independent work here
try {
    auto result = future.get();
} catch (const std::exception& e) {
    // exception is rethrown from task
}
```

- Work stealing havuzları dengesiz görev dağılımında faydalıdır.
- Bounded queue backpressure sağlayarak belleğin sınırsız büyümesini önler.
- Task exceptionları kaybolmamalı; future veya merkezi hata kanalıyla taşınmalıdır.
- Shutdown: yeni iş kabulünü kapat, kuyruk politikasını belirle, çalışanları güvenli durdur.

Mini tasarım

4 worker threadli, en fazla 1000 bekleyen iş alan bounded queue thread pool için durum makinesini çizin.

MODÜL 11

SIMD ve Data-Oriented Design

Data-Oriented Design, sınıf hiyerarşisinden çok verinin işlemci üzerinde nasıl tüketildiğini merkeze alır. Structure of Arrays (SoA), aynı alan üzerinde toplu işlem yapılan senaryolarda cache ve SIMD açısından Array of Structures (AoS) yaklaşımından avantajlı olabilir.

AOS VS SOA

```
// AoS
struct Particle { float x, y, z, mass; };
std::vector<Particle> particles;

// SoA
struct Particles {
    std::vector<float> x, y, z, mass;
};

// Compiler may auto-vectorize contiguous loops
for (size_t i=0; i<p.x.size(); ++i)
    p.x[i] += velocity * dt;
```

Teknik	Hedef
SoA	Aynı alanların ardışık bellekte tutulması
SIMD	Tek komutla çok veri ögesi
Loop unrolling	Branch/loop overhead azaltma potansiyeli
Branch reduction	Pipeline tahmin kaybını azaltma

Ölçüm

SoA her zaman kazanmaz. Nesnenin tüm alanları beraber kullanılıyorsa AoS daha doğal ve cache dostu olabilir.

MODÜL 12

Profiling, Flame Graph ve Sanitizer

Profiler hangi fonksiyonlarda zaman harcadığını; hardware counter araçları cache miss, branch miss ve instruction davranışını gösterir. Sanitizerlar performanstan önce doğruluk için kritiktir: veri yarışı veya use-after-free bulunan kodun benchmark sonucu anlamlı değildir.

Araç sınıfı	Örnek amaç
Sampling profiler	Hot function ve call stack
Hardware counters	Cache/branch/instruction davranışı
ASan/UBSan	Bellek hatası ve undefined behavior
TSan	Data race tespiti

ARAÇ KOMUTLARI

```
# Release build + profiling
g++ -O3 -DNDEBUG -march=native app.cpp -o app
perf record -g ./app
perf report

# Sanitizers (compiler dependent)
-fsanitize=address,undefined
-fsanitize=thread
```

Optimizasyon döngüsü

1) Baseline ölç. 2) Profiler ile hotspot bul. 3) Tek hipotez oluştur. 4) Değiştir. 5) Aynı koşulda yeniden ölç. 6) Doğruluk testlerini çalıştır.

MODÜL 13 - FINAL

Bitirme Projesi: Yüksek Hızlı Telemetry İşleyici

Senaryo: Çok sayıda sensör olayı belleğe geliyor. Sistem saniyede yüksek mesaj sayısını düşük p99 gecikme ile işleyecek, her sensör için sayaç ve hareketli istatistik tutacak ve periyodik snapshot üretecek.

Proje gereksinimleri

- Baseline: tek thread, vector tabanlı veri yapısı ve ölçülen throughput/p50/p99.
- Mesaj yapısının sizeof/alignment değerlerini raporla; gerekirse alan düzenini değerlendir.
- Allocation sayısını azaltmak için reserve veya pmr deneyi yap.
- En az iki concurrency tasarımı karşılaştır: mutex korumalı kuyruk ve alternatif task/atomic yaklaşımı.
- Thread ölçeklenmesini 1/2/4/8 thread ile ölç.
- Profiler çıktısında ilk 5 hotspotu kaydet.
- ASan/UBSan; concurrency sürümünde mümkünse TSan çalıştır.
- Son raporda her optimizasyonun yüzde etkisini ve karmaşıklık maliyetini yaz.

Kısa değerlendirme

Soru	Beklenen odak
1. False sharing nedir?	Aynı cache line için coherence trafiği
2. relaxed ne garanti etmez?	Threadler arası ordering
3. reserve neyi azaltabilir?	Reallocation/allocation
4. SoA neden hızlanabilir?	Locality + SIMD
5. En önemli optimizasyon kuralı?	Önce ölç, sonra değiştir

Başarı ölçütü

Proje yalnızca daha hızlı olmamalı; ölçüm tekrarlanabilir, thread safety doğrulanmış ve optimizasyonların karmaşıklık karşılığı açıkça belgelenmiş olmalıdır.